

Week 9: RAG & Mixture of Experts

Enhancing LLMs with Retrieval and Routing 

LLM Course

Today's Journey [?]

1. [?] **Retrieval Augmented Generation (RAG):** Grounding LLMs in external knowledge
2. [?] **Mixture of Experts (MoE):** Efficient scaling through specialization
3. [?] **Self-Supervised Learning:** SimCLR and contrastive methods
4. [?] **Multimodal Learning:** CLIP and vision-language alignment
5. [?] **What's Next for LLMs?:** Future directions and open problems
6. [?] **Final Project:** Guidelines and inspiration

Goal: Look toward the future while grounding ourselves in current techniques

Retrieval Augmented Generation

[?] Discussion Question

What are the limits of parametric memory?

Consider: An LLM's knowledge is frozen at training time...

Problems with purely parametric models:

- [?] **Knowledge cutoff:** No information after training
- [?] **Hallucinations:** Models confidently generate false information
- [?] **No source attribution:** Can't cite where information comes from
- [?] **Expensive updates:** Retraining for new information is costly
- [?] **Privacy concerns:** Sensitive data baked into parameters
- [?] **Domain specificity:** Limited knowledge of specialized domains

Solution: Combine parametric knowledge with non-parametric retrieval [?]

What is RAG?

Retrieval Augmented Generation

A technique that enhances LLMs by retrieving relevant documents from an external knowledge base and using them to inform generation.

Key Idea: Instead of relying only on learned parameters, the model can "look things up"!

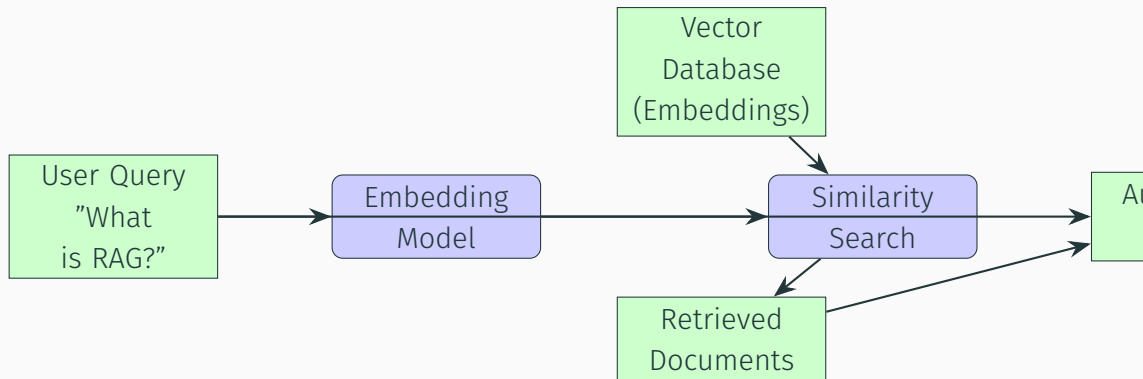
Traditional LLM:

- Question → Model → Answer
- Only parametric knowledge
- Fixed at training time

RAG Pipeline:

- Question → Retrieve Docs
- Docs + Question → Model
- Grounded Answer

Reference: Lewis et al. (2020) - "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks"



Steps:

1. Embed user query
2. Search vector database for similar documents

RAG Implementation Example ?

Basic RAG with HuggingFace and ChromaDB:

```
1 from langchain.vectorstores import Chroma
2 from langchain.embeddings import HuggingFaceEmbeddings
3 from langchain.llms import HuggingFacePipeline
4 from langchain.chains import RetrievalQA
5
6 # 1. Create vector database from documents
7 embeddings = HuggingFaceEmbeddings(model_name="sentence-transformers/all-MiniLM-L6-v2")
8 vectordb = Chroma.from_documents(
9     documents=docs,
10     embedding=embeddings,
11     persist_directory="./chroma_db"
12 )
13
14 # 2. Set up retriever
15 retriever = vectordb.as_retriever(
16     search_type="similarity",
17     search_kwargs={"k": 3} # Retrieve top 3 documents
18 )
19
20 # 3. Create RAG chain
21 llm = HuggingFacePipeline.from_model_id(
22     model_id="meta-llama/llama-2-7b-chat-hf",
23     task="text-generation"
24 )
25
```

Self-RAG (Asai et al., 2023)

- Model decides *when* to retrieve
- Self-reflects on retrieval quality
- Critique and refine generations
- More adaptive and efficient

Key Idea: Not every query needs retrieval!

Corrective RAG (Yan et al., 2024)

- Evaluates retrieved documents
- Filters irrelevant content
- Re-retrieves if needed
- Decomposes complex queries

Key Idea: Quality over quantity in retrieval!

Evolution of RAG

Basic RAG → Self-RAG → Corrective RAG → Agentic RAG

The field is moving toward more autonomous, self-correcting systems!

References: Asai et al. (2023) - Self-RAG; Yan et al. (2024) - Corrective RAG

1. Document Processing

- Chunking strategies (fixed, semantic, recursive)
- Metadata extraction
- Deduplication

2. Embedding Models

- Dense retrieval: Sentence-BERT, E5, BGE
- Sparse retrieval: BM25, TF-IDF
- Hybrid approaches

3. Vector Databases

- FAISS, ChromaDB, Pinecone, Weaviate, Qdrant
- Indexing strategies (IVF, HNSW)
- Approximate nearest neighbor search

4. Prompt Engineering

- How to format retrieved context
- Instruction templates

Comparing RAG Approaches ?

Approach	When to Retrieve	How to Filter	Best For
Naive RAG	Always	Top-k similarity	Simple Q&A, prototyping
Self-RAG	Model decides	Self-reflection tokens	Adaptive retrieval needs
Corrective RAG	Always	Relevance scoring	High-precision requirements
HyDE	Always (on generated answer)	Similarity to hypothesis	Complex reasoning
Agentic RAG	Tool-based decision	Multi-step filtering	Complex workflows

Real-world considerations:

Challenges:

-  Cost: Embedding + storage + inference
-  Latency: Retrieval adds overhead
-  Context window limits
-  Retrieval quality issues
-  Keeping data fresh
-  Security and privacy

Solutions:

- Caching embeddings
- Semantic caching
- Incremental indexing
- Re-ranking strategies
- Smaller, efficient models
- Hybrid search methods

Production Tip

Start simple (Naive RAG), measure performance, then optimize based on actual

Mixture of Experts

? Discussion Question

Larger models are better... but training 100B+ parameter models is:

- Computationally expensive ?
- Energy intensive ?
- Slow to train ?
- Slow at inference ?

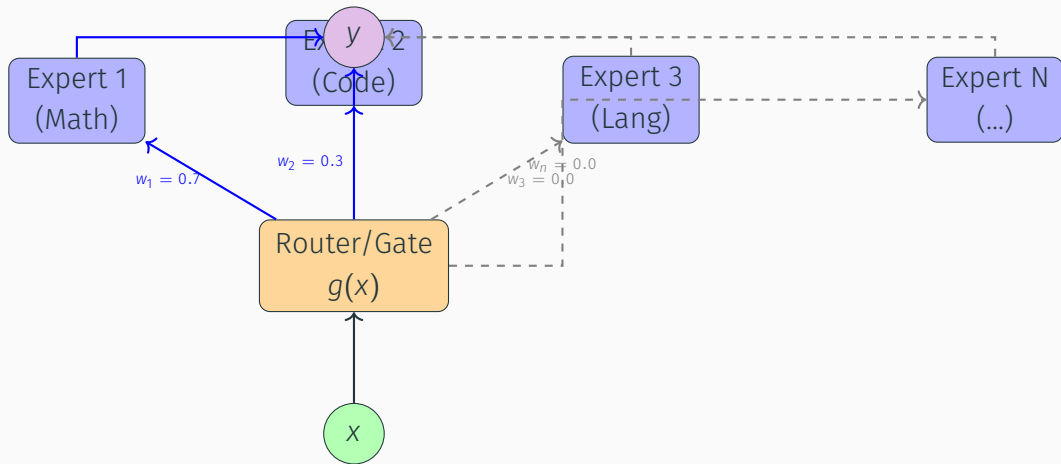
Can we get the benefits of scale without the costs?

Key Insight: Not all parameters need to be active for every input!

Mixture of Experts (MoE)

A neural network architecture where different "expert" sub-networks specialize in

MoE Architecture [?]



$$y = \sum_{i=1}^N g(x)_i \cdot E_i(x)$$

Routing Mechanism:

1. For input x , router computes scores: $h(x) = W_g \cdot x$
2. Apply softmax to get probabilities: $g(x) = \text{softmax}(h(x))$
3. Select top- k experts (typically $k = 2$)
4. Route token only to selected experts
5. Aggregate outputs: $y = \sum_{i \in \text{top-}k} g(x)_i \cdot E_i(x)$

Benefits:

- ? **Sparse computation:** Only 2 out of N experts active
- ? **Scalable:** Can add more experts without increasing per-token cost
- ? **Specialization:** Experts learn different capabilities
- ? **Efficiency:** Better quality/compute ratio

Challenges:

1. Load Balancing

- Some experts overused
- Others underutilized

2. Training Instability

- Routing collapse
- Expert specialization

3. Memory Requirements

- All experts in memory
- High bandwidth needs

Solutions:

1. Load Balancing Loss

- Auxiliary loss term
- Encourages uniform usage

2. Expert Capacity

- Limit tokens per expert
- Dynamic capacity buffers

3. Expert Parallelism

- Distribute experts across GPUs
- Optimize communication

Key Innovation

Load balancing loss: $\mathcal{L}_{\text{balance}} = \alpha \cdot \sum_{i=1}^N f_i \cdot P_i$

where f_i is the fraction of tokens routed to expert i , and P_i is its probability.

Mixtral 8x7B: MoE in Action

Mixtral 8x7B (Jiang et al., 2024)

A state-of-the-art sparse mixture of experts model with 8 experts, each 7B parameters.

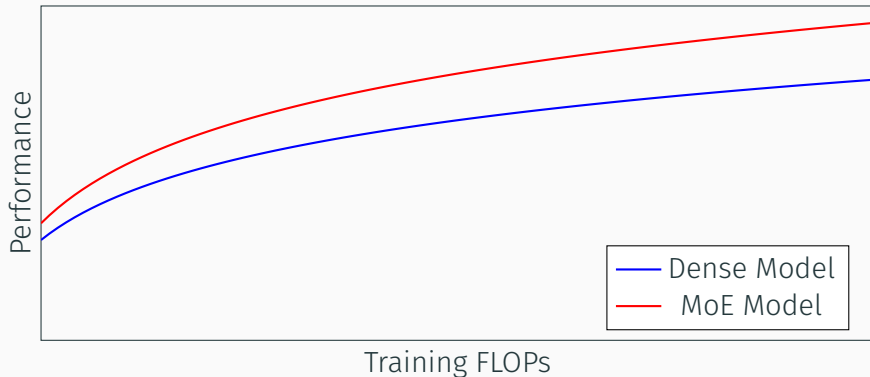
Architecture:

- **Total parameters:** 47B (8 experts $\times$ 7B, minus shared layers)
- **Active parameters:** 13B (only 2 experts active per token)
- **Performance:** Matches or exceeds Llama 2 70B
- **Speed:** 6 $\times$ faster inference than dense 70B model

Key Results:

Model	Active Params	MMLU	Speed
Llama 2 70B	70B	69.7	1 $\times$
Mixtral 8x7B	13B	70.6	6 $\times$

MoE vs Dense Models ?



Trade-offs:

MoE Advantages:

- Better compute efficiency

Faster training

Dense Advantages:

- Simpler architecture

Easier to train

Self-Supervised & Contrastive Learning

Self-Supervised Learning

Learning representations from unlabeled data by creating pretext tasks where the data itself provides the supervision signal.

Why is this important?

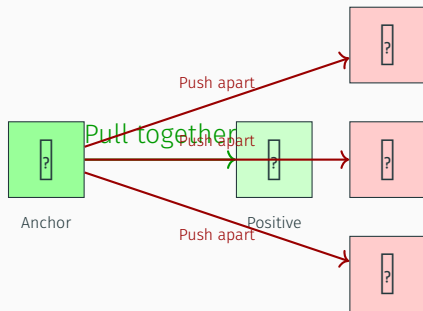
- Labeled data is expensive and scarce
- Unlabeled data is abundant
- Learn general-purpose representations
- Transfer to downstream tasks

Examples of pretext tasks:

- **Language:** Masked language modeling (BERT), next token prediction (GPT)
- **Vision:** Predicting rotations, colorization, jigsaw puzzles
- **Contrastive:** Making similar things close, dissimilar things far apart

Contrastive Learning [?]

Core Idea: Learn by contrasting positive pairs against negative pairs



Objective: $\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\sum_{j=1}^N \exp(\text{sim}(z_i, z_j)/\tau)}$

where z_i is anchor, z_i^+ is positive, and $\{z_j\}$ includes negatives.

SimCLR: Simple Contrastive Learning

SimCLR (Chen et al., 2020)

"A Simple Framework for Contrastive Learning of Visual Representations"

Method:

1. Take an image
2. Apply two different random augmentations (crop, color, blur, etc.)
3. Encode both views with a neural network
4. Pull representations of the two views together
5. Push apart from other images in the batch

Key Innovations:

- Strong data augmentation is crucial
- Learnable projection head improves performance
- Large batch sizes help (more negatives)
- Longer training – better representations

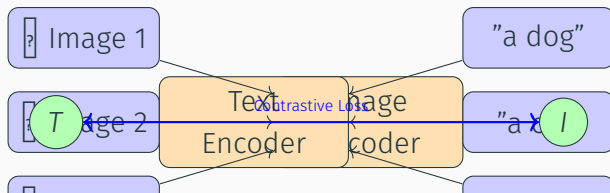
CLIP: Connecting Vision & Language

CLIP (Radford et al., 2021)

"Learning Transferable Visual Models From Natural Language Supervision"

Big Idea: Train vision and language encoders together using contrastive learning!

	T_1	T_2	T_3
I_1		x	x
I_2	x		x
I_3	x	x	



Training Data: 400 million (image, text) pairs from the internet

Training Objective:

- Given a batch of N (image, text) pairs
- Maximize similarity for N correct pairs
- Minimize similarity for $N^2 - N$ incorrect pairs
- Symmetric contrastive loss

Applications:

- **Zero-shot classification**
 - No training on target dataset!
 - Competes with supervised models
- **Image retrieval**
- **Foundation for generative models**
 - DALL-E, Stable Diffusion
 - Text-guided editing
- **Multimodal reasoning**

What's Next for LLMs?

Where we are in 2024:

- ? Strong language understanding and generation
- ? Multimodal capabilities (text, image, audio)
- ? Long context windows (100k+ tokens)
- ? Efficient architectures (MoE, state space models)
- ? Improved safety and alignment

But still limitations:

- ? Reasoning on complex, multi-step problems
- ? Factual accuracy and hallucinations
- ? Limited "world models" and common sense
- ? Lack of true planning and goal-directed behavior
- ? No persistent memory across sessions

Future Directions

Beyond text: Unified models that understand and generate across all modalities

Current trends:

- **Vision-Language Models:** GPT-4V, Gemini, Claude 3
- **Audio:** Whisper, AudioLM, MusicLM
- **Video:** VideoLLM, video understanding
- **3D/Spatial:** Scene understanding, robotics

Research questions:

- How to effectively align different modalities?
- What's the right architecture for true multimodality?

? Future Directions

Moving beyond pattern matching: True reasoning, planning, and problem-solving

Current approaches:

- **Chain-of-Thought:** Prompting for step-by-step reasoning
- **Tree-of-Thoughts:** Exploring multiple reasoning paths
- **Tool Use:** Augmenting with calculators, search, code execution
- **Self-Consistency:** Sampling multiple answers and voting
- **Formal Verification:** Connecting to theorem provers

Open challenges:

- Abstract reasoning (ABC benchmark)

? Future Directions

LLMs as autonomous agents: Acting in environments, pursuing goals, learning from feedback

Components of LLM agents:

1. **Perception:** Understanding environment state
2. **Planning:** Decomposing goals into sub-tasks
3. **Action:** Using tools and APIs
4. **Memory:** Short-term and long-term context
5. **Learning:** Improving from feedback

Applications:

- Personal assistants that manage your tasks

Future Directions

Democratizing AI: Making powerful models accessible and sustainable

Efficiency innovations:

Model-level:

- Mixture of Experts
- Quantization (4-bit, 3-bit)
- Knowledge distillation
- Sparse models
- State space models (Mamba)

System-level:

- Better hardware (TPUs, GPUs)
- Flash attention
- KV cache optimization
- Speculative decoding
- Model parallelism

Environmental considerations:

? Future Directions

LLMs that know you: Persistent context, preferences, and relationships

Challenges:

- Current models are stateless (reset every conversation)
- Limited context windows
- Privacy concerns with personal data
- How to update/forget information?

Potential solutions:

- **RAG over personal data:** Your documents, emails, notes
- **Fine-tuning on user data:** Personal writing style
- **Memory modules:** Explicit long-term memory storage

Speculative: What Could Go Wrong?

Technical Risks:

- Scaling plateaus
- Inability to solve core reasoning
- Data quality degradation
- Model collapse from synthetic data
- Unforeseen safety issues

Societal Risks:

- Job displacement
- Misinformation at scale
- Concentration of power
- Privacy violations
- Alignment failures
- Autonomous weapon systems

Discussion Question

Discussion: How do we ensure AI development benefits humanity? What role do researchers, companies, and governments play?

We're still in the early days!

- Most low-hanging fruit has been picked, but huge opportunities remain
- Fundamental questions are still open
- New architectures are being discovered
- Applications are still being invented
- The field is moving incredibly fast

This is YOUR opportunity to contribute!

Think About It

What problem in AI excites you most?

Final Project

The Capstone

Your final project is a chance to explore a topic deeply, conduct original research or build something novel, and demonstrate what you've learned!

Project Types:

1. **Research Project:** Investigate a question, run experiments, analyze results
2. **Application Project:** Build a working system that solves a real problem
3. **Reproduction Study:** Replicate a recent paper and extend it
4. **Critical Analysis:** Deep dive into a method's limitations and propose improvements

Team Formation:

- Work individually or in teams of 2-3
- Diverse skill sets are valuable!

Project Timeline

Date	Milestone	Deliverable
Week 9	Team Formation	Form teams, brainstorm ideas
Week 10	Proposal	2-page proposal with plan
Week 11-12	Development	Work on project, check-ins
Week 13	Draft Results	Preliminary findings
Week 14	Presentations	Present to class (15 min)
Finals Week	Final Report	8-10 page report

Grading Breakdown:

- Proposal: 10%
- Presentation: 30%
- Final Report: 50%
- Novelty/Impact: 10%

1. Domain-Specific RAG Systems

- Build RAG for medical literature, legal documents, or technical docs
- Compare embedding models and retrieval strategies
- Evaluate accuracy and source attribution

2. Advanced RAG Techniques

- Implement and compare Self-RAG vs. Corrective RAG
- Explore hybrid retrieval (sparse + dense)
- Study the effect of chunk size and overlap

3. RAG for Multimodal Content

- Retrieve images, diagrams, or videos along with text
- Use CLIP embeddings for visual retrieval
- Generate answers that reference visual content

4. Mixture of Experts Analysis

- Study how experts specialize in Mixtral or similar models
- Visualize routing decisions
- Experiment with different routing mechanisms

5. Efficient Fine-tuning

- Compare LoRA, QLoRA, prefix tuning, and full fine-tuning
- Study the performance/efficiency trade-off
- Apply to a specific task

6. State Space Models

- Investigate Mamba or other SSM architectures
- Compare to Transformers on long sequences
- Hybrid architectures

7. Reasoning Enhancement

- Implement Tree-of-Thoughts or Graph-of-Thoughts
- Benchmark on reasoning tasks (GSM8K, ARC, etc.)
- Analyze failure modes

8. LLM Agents

- Build an agent for a specific task (coding, research, planning)
- Implement tool use and memory
- Evaluate on complex multi-step tasks

9. Interactive Learning

- Study learning from human feedback
- Implement RLHF or DPO

Explore alternatives to RLHF

10. Vision-Language Models

- Fine-tune CLIP for a specific domain
- Build visual question answering system
- Explore compositional understanding

11. Text-to-X Generation

- Text-to-image, text-to-music, text-to-3D
- Study controllability and quality
- Novel conditioning mechanisms

12. Creative Applications

- Story generation with consistency
- Code generation and debugging
- Scientific writing assistant

13. Model Interpretability

- Probe what models learn at different layers
- Visualize attention patterns
- Study emergence of capabilities

14. Bias & Fairness

- Measure and mitigate biases in LLMs
- Study fairness across demographics
- Propose debiasing techniques

15. Robustness & Safety

- Adversarial attacks on LLMs
- Jailbreak detection and prevention
- Output filtering and safety layers

Making Your Project Stand Out

1. Clear Research Question

- What specific question are you answering?
- Why does it matter?

2. Rigorous Evaluation

- Use established benchmarks
- Compare to baselines
- Statistical significance

3. Insightful Analysis

- Don't just report numbers
- Understand WHY things work or fail
- Error analysis and case studies

4. Clear Communication

- Write for your peers
- Good visualizations
- Reproducible results

Computational Resources:

- Google Colab (free tier + Pro)
- HuggingFace Spaces
- University compute cluster (contact instructor)
- Free API credits (OpenAI, Anthropic, etc.)

Getting Help:

- Office hours: Weekly check-ins on progress
- Discord: #final-project channel
- Peer review: Exchange proposals with other teams
- Instructional team: We're here to help!

Code & Data:

HuggingFace Hub: Pre-trained models, datasets

Your 2-page proposal should include:

1. **Title & Team Members**
2. **Problem Statement** (1 paragraph)
 - What problem are you solving?
 - Why is it important?
3. **Related Work** (1 paragraph)
 - What has been done before?
 - How is your work different?
4. **Approach** (1-2 paragraphs)
 - What methods will you use?
 - What data/models will you need?
5. **Evaluation Plan** (1 paragraph)
 - How will you measure success?
 - What metrics and baselines?
6. **Timeline** (bullet points)

Start Early!

The best projects come from iteration and refinement. Don't wait until the last minute!

Do:

-  Pick a topic you're genuinely interested in
-  Start with a simple baseline and iterate
-  Keep a research log of what you try
-  Ask for feedback early and often
-  Budget time for debugging and writing

Don't:

-  Pick something too ambitious for the timeframe
-  Rely on things you can't access (massive compute, proprietary data)

Wrap-Up

Key Takeaways ?

1. **RAG** extends LLMs with external knowledge
 - Solves knowledge cutoff and hallucination problems
 - Many variations: Self-RAG, Corrective RAG, Agentic RAG
2. **Mixture of Experts** enables efficient scaling
 - Sparse activation → better compute efficiency
 - Successful in practice (Mixtral, GPT-4 likely uses MoE)
3. **Self-supervised learning** powers modern AI
 - Contrastive methods align modalities (CLIP)
 - Unlocks learning from unlabeled data
4. **The future is multimodal, agentic, and efficient**
 - Many open problems remain
 - You can contribute!

Required:

1. **Lewis et al. (2020)**: Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks [arXiv]
2. **Shazeer et al. (2017)**: Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer [arXiv]
3. **Radford et al. (2021)**: Learning Transferable Visual Models From Natural Language Supervision (CLIP) [arXiv]

Recommended:

- Asai et al. (2023): Self-RAG [arXiv]
- Yan et al. (2024): Corrective RAG [arXiv]
- Jiang et al. (2024): Mixtral of Experts [arXiv]
- Chen et al. (2020): SimCLR [arXiv]
- HuggingFace Blog: MoE [Blog]

Action Items for This Week

1. Read the required papers
2. Form final project teams
3. Brainstorm project ideas
4. Start working on proposal (due Week 10)
5. Experiment with RAG or MoE (optional but encouraged!)

This is the final stretch—make it count!

? Think About It

What's the one thing from this course that will stick with you?

Questions?

Good luck with your final projects!

 Contact: [Your Email]

 Discord: [Your Discord]

 Office Hours: [Your Schedule]